

zero_shortcomings_audit

scripts/

zero_shortcomings_audit.sh

Revision: 4 **Last modified:** 2026-09-26T08:50:32Z

Purpose: Unified enumeration, closure-evidence re-verification, and standing-check entry point across this project’s three tracked “unfinished/gap/shortcoming” surfaces (open backlog items, unimplemented named gates, open coverage-escapes). See specs/003-zero-shortcomings-audit/contracts/cli.md for the full CLI contract.

Usage: scripts/zero_shortcomings_audit.sh <enumerate|verify-closure|standing-check> [options]

-h / --help prints the usage text and exits 0. Running with no mode prints the usage and exits 1. An unknown mode prints unknown mode: <x> plus the usage and exits 1. (The built-in --help text lists every mode and option; this guide gives the full semantics.)

Modes

enumerate [--json] [--surface S] [--sort-by-risk]

Read-only. S is one of all (default), backlog, gates, escapes, blocked.

- Default text output: Open backlog items, Unimplemented gates, Open coverage-escapes counts (only for the selected surface).
- --json: one JSON object with the keys backlog_open, gates_unimplemented, escapes_open (only the selected surfaces’ keys).
- --surface blocked: lists every Operator-blocked item as id|unblock_condition (left-joined from operator_block_details, ordered by id), never a bare count. An Operator-blocked item with

no details row, or with a blank/whitespace-only condition, is listed as `id|MISSING-UNBLOCK-CONDITION` (never dropped) and a `WARN: N Operator-blocked item(s) lack an unblock condition` line goes to `stderr`; the exit status stays 0 (enumerate reports findings, it does not gate on them).

- `--sort-by-risk` (FR-012): prints the open backlog item ids ordered by reopen count descending (count of Reopened events in `item_history`), then `last_modified` descending, then `id`. With `--json` it prints a JSON array of ids. **Only valid with `--surface backlog`.**
- Errors (all exit 1, message on `stderr`): the tracker DB (backlog/blocked surfaces) is missing or unreadable (tracker DB missing or unreadable: `<path>` – checked before any `sqlite3` call, so a missing DB is never created as an empty file); `--surface` given with no value (enumerate: `--surface` requires a value ...); unknown option (enumerate: unknown option: X); unrecognized `--surface` value (enumerate: unrecognized `--surface` value: 'X' (expected all|backlog|gates|escapes|blocked)); `--sort-by-risk` with any surface other than backlog (enumerate: `--sort-by-risk` applies only to `--surface backlog` (got 'X'))).
- The gate count fails loud (exit 1, message naming the gate script) when the gate-ledger script prints no parseable `unimplemented=N` line, rather than reporting 0.

verify-closure <item-id> [--reopen-on-mismatch] [--require-layer L]

Independently re-runs the recorded closure evidence for one item. Evidence is read from the first `closure_evidence_*.md` directly inside `<AUDIT_QA_ROOT>/<item-id>/`. `<item-id>` must match `^[A-Za-z0-9][A-Za-z0-9._-]*$` and must not contain `..`; anything else (a `/`, a leading `-` or `..`, spaces, shell metacharacters) is rejected with `invalid item id` before the id is used in any path. `--require-layer L` (`source|artifact|runtime`, default `runtime`) is the minimum evidence layer accepted; the evidence's `**Evidence Layer:**` field (default `source` when absent) must rank at or above it (`source < artifact < runtime`; an unrecognized label ranks as `source`).

Required evidence-file fields:

- `**Evidence Layer:**` (optional; see above)
- `**Test Type:**` (FR-010) – required. Allowed values, compared case-insensitively after trimming: `unit`, `integration`, `e2e`, `security`, `stress`, `chaos`, `scaling`, `ui`, `challenge`.

- ****Command:**** `<command>` – the command to re-run. It runs as `bash -c "<command>"` in a **fresh process** with the working directory set to the repository root (the one this script lives in, not the caller's cwd) and stdin closed, so it does not inherit this script's `set -euo pipefail`, functions or non-exported variables and gives the same result wherever `verify-closure` is invoked from. Exported environment variables are inherited, as for any child process. The command's stderr is shown on the terminal only after passing through the redactor (see Redaction).
- ****Result Summary:**** `<text>` – compared exactly against the command's fresh stdout.

Trust model: the ****Command:**** is executed as arbitrary shell. Evidence files are trusted, tracked input – review them like code; never point `AUDIT_QA_ROOT` at untrusted files.

Exit codes:

Code	Meaning
0	The recorded command's fresh output equals the recorded Result Summary (reproduced its recorded evidence).
1	Mismatch (MISMATCH -- recorded '...', got '...', values credential-redacted before printing); or usage error: missing or invalid item id, unknown option, -- require-layer with no value or a value other than source/artifact/runtime; or the evidence snapshot / a temp file could not be created (the command is then not run).
2	No <code>closure_evidence_*.md</code> found; declared layer below --require-layer; no **Test Type:** ; invalid **Test Type:** ; no **Command:** field.
3	Mismatch and --reopen-on-mismatch was given and the tracker reopen failed (reopen

Code	Meaning
	FAILED for <id> -- the mismatch is NOT recorded in the tracker).

--reopen-on-mismatch: on a mismatch, calls the workable-items reopen CLI (--why test-failed --who AI, evidence path = the evidence file, DB = WORKABLE_ITEMS_DB_OVERRIDE or docs/workable_items.db) and returns 1 when it succeeds. A failing reopen is reported with exit 3, never swallowed.

Evidence-corruption guard (FR-008): before running the recorded command the script snapshots (content hashes) every git-tracked file under docs/qa/ of the repository root; afterwards any such file **modified or deleted** outside <AUDIT_QA_ROOT>/<item-id>/ is restored (files that were already dirty before the run are restored from a byte copy of their pre-command content, others via git checkout --) and the incident is appended to <AUDIT_QA_ROOT>/zero_shortcomings_audit/<run-id>.log (or \$AUDIT_INCIDENT_LOG_DIR/<run-id>.log) with a warning on the console. The guard never changes the exit code by itself.

Guard limits (stated, not hidden):

- The snapshot is taken immediately before and compared immediately after the command. A **legitimate** edit made by **another process** to a tracked docs/qa/ file during that window is indistinguishable from corruption and **will be reverted**. Run verify-closure sequentially, never alongside other writers of docs/qa/.
- Paths with spaces or tabs are handled; a tracked path containing a **newline** cannot be snapshotted (git hash-object --stdin-paths is newline-delimited) and is skipped with a WARN naming it.
- Files the command **creates** (untracked) and changes inside the item's own evidence directory are left alone.

standing-check

Recurring, advisory mode; **always exits 0**. Runs enumerate --json for each of the backlog, gates, escapes surfaces and appends one line to <AUDIT_STANDING_LOG_DIR>/<run-id>.log:

```
<run-id> mode=standing-check status=ok
{"backlog_open":N,"gates_unimplemented":N,"escapes_open":N}
```

If a sub-check fails, the line instead carries `status=degraded` `reason=<list>` `failed=<list>` (reasons such as `db_missing`, `<surface>_rc<N>`) and a `WARN` is printed; the JSON body then holds only the surfaces that succeeded. A missing/unreadable `docs/workable_items.db` is detected before any `sqlite3` call (so a missing file is never silently created as an empty DB) and recorded as `db_missing`. If redaction itself fails the line falls back to `status=degraded` `reason=redaction_failed`.

In `scripts/pre_build_verification.sh` (stage [59/59] `CM-ZERO-SHORTCOMINGS-STANDING`) a `status=ok` run counts as a `PASS`; a `status=degraded` run, a non-zero exit, or a failure to create the stage's temp files each print a `WARN` line and are **not** counted as a `PASS`. The stage never calls `fail()` and never changes the sweep's exit status.

Run id format: `YYYYMMDDTHHMMSSZ-pid<PID>`.

Environment overrides

Variable	Default	Effect
<code>WORKABLE_ITEMS_DB_OVERRIDE</code>	<code>docs/workable_items.db</code>	Backlog / blocked / reopen SQLite DB path.
<code>GATE_LEDGER_SCRIPT_OVERRIDE</code>	<code>constitution/scripts/gates/cm_gate_ledger_ratchet.sh</code>	Gate-ledger script used for the unimplemented-gate count.
<code>AUDIT_QA_ROOT</code>	<code>docs/qa</code>	Root searched for <code><item-id>/closure_evidence_*.md</code> .
<code>AUDIT_STANDING_LOG_DIR</code>	<code>docs/qa/zero_shortcomings_audit</code>	Directory for the standing-check log (created if missing).
<code>AUDIT_INCIDENT_LOG_DIR</code>	<code><AUDIT_QA_ROOT>/zero_shortcomings_audit</code>	Directory for verify-closure corruption-incident logs.

Redaction (Constitution Principle III / §11.4.10)

Every value printed on a mismatch, the recorded command's stderr, every command quoted in a corruption incident, and every standing-check log line pass through `audit_redact_before_write`. Values are replaced by `<redacted-per-§11.4.10>`; names stay loggable. Comparison itself uses the raw values; only what is printed or written is redacted.

Keywords (case-insensitive, matched anywhere in a name, e.g. `RUTRACKER_PASSWORD`): `password`, `passwd`, `secret`, `api_key` (also `api-key`/`apikey`), `access_token`, `auth_token`, `client_secret`, plus the project extension `key`, `token`, `cookies`.

The rules run in order, one line at a time:

1. `*_COOKIES=` / `Cookie:` – everything to the end of the line (cookie strings contain spaces).
2. `Authorization:` – everything to the end of the line.
3. URL userinfo `scheme://user:PASSWORD@host` – the password (the user stays); a token-only `scheme://TOKEN@host` (GitHub PAT clone form) – the whole token; and `curl -u user:PASSWORD` – the password.
4. `Bearer <token>` / `Basic <token>` – the token.
5. `<keyword>["'"]?<sep>"double-quoted value"` – the whole quoted value (JSON style included).
6. The same with single quotes.
7. `<keyword>["'"]?<sep>value` – the unquoted token (`sep` = `:` or `=`, spaces allowed).
8. A space-separated CLI flag `--<...keyword> value`.

Known limits: an escaped quote inside a quoted value (`"a\"b"`) ends the match early; a secret split across lines, a secret with no recognisable keyword before it (a bare token or base64 blob), and a value introduced by any other separator are **not** redacted. Bare key/token/bearer favour recall, so some non-secret text is over-redacted by design.

Inputs: `docs/workable_items.db` (read-only), `constitution/scripts/gates/` (read-only, via `cm_gate_ledger_ratchet.sh`), `docs/QA_DISCOVERY_LEDGER.md` (read-only), `docs/qa/<item-id>/closure_evidence_*.md` (read; verify-closure).

Outputs: A human-readable or --json report; standing-check appends one line to <AUDIT_STANDING_LOG_DIR>/<run-id>.log; verify-closure appends to <AUDIT_QA_ROOT>/zero_shortcomings_audit/<run-id>.log (or \$AUDIT_INCIDENT_LOG_DIR) only when the corruption guard reverted something.

Side-effects: None in enumerate. standing-check writes its log line. verify-closure executes the recorded command (bash -c, fresh process, cwd = repo root), may revert corrupted tracked evidence files, and with --reopen-on-mismatch may call the workable-items CLI to reopen an item.

Dependencies: sqlite3, git, constitution/scripts/workable-items/workable-items, constitution/scripts/gates/cm_gate_ledger_ratchet.sh, and scripts/lib/audit_*.sh.

Cross-references: specs/003-zero-shortcomings-audit/{spec,plan,research,data-model}.md.

Last verified: 2026-09-26 (against the script source and the tests/audit/ suite; the --help text itself was run).